PROJECT PLAN

Network File Transfer with Sender, Receiver & Hash Checker

Python • TCP Networking • SHA-256 Integrity Verification

Purpose	Transfer files between computers and verify that received contents match the originals.
Key requirement	Multiple files may share the same filename; each must be stored and checked independently.
Core technologies	Python standard library: socket, hashlib, json, struct, pathlib, uuid, sqlite3, logging.

1. Project Overview

The system has three main modules. The Sender selects files, generates a unique transfer ID, calculates the SHA-256 digest, and sends metadata plus file bytes. The Receiver accepts the transfer over TCP, stores the file safely, and calculates its own digest. The Hash Checker compares the sender and receiver digests and reports VERIFIED or MISMATCH. It supports automatic checks after transfer and manual checks later.

2. Architecture and Workflow

SENDER	NETWORK / TCP	RECEIVER	HASH CHECKER
Select file(s) Generate UUID Calculate SHA-256	Length-prefixed metadata File size + file bytes Reliable stream reads	Validate metadata Receive exact byte count Save in unique folder	Recalculate SHA-256 Compare expected digest Store and display result

Workflow: Start receiver → select files in sender → create a UUID per file → calculate sender hash → send metadata and bytes → save received file under its UUID → calculate receiver hash → compare → record result → allow manual re-checking.

3. Module Responsibilities

Module / file	Responsibilities
Sender — sender.py	Select files; calculate SHA-256; create a unique transfer ID; send metadata and file content; show progress and errors.
Receiver — receiver.py	Listen on a configured host/port; accept connections; validate metadata; receive exact file size; save safely without overwriting.
Hash Checker — hash_checker.py	Automatically compare hashes after transfer; support manual file selection; show VERIFIED, MISMATCH, or FAILED.
Protocol — protocol.py	Define message framing, length-prefixed metadata, exact TCP reads, and chunked file transfer.
Hashing — hash_utils.py	Calculate SHA-256 in chunks so large files do not need to be loaded entirely into memory.

Module / file	Responsibilities
Storage — storage.py	Use SQLite to store transfer IDs, original names, file sizes, expected hashes, paths, timestamps, and verification status.
Configuration — config.py	Keep host, port, destination folder, and allowed size limits configurable.

4. Folder Structure

SecureFileTransfer/
■■■ sender.py
■■■ receiver.py
■■■ hash_checker.py
■■■ protocol.py
■■■ hash_utils.py
■■■ storage.py
■■■ config.py
■■■ requirements.txt
■■■ README.md
■■■ sender_files/ (test files to send)
■■■ received/ (one UUID folder per transfer)
■■■ data/transfers.db
■■■ logs/transfer.log
■■■ tests/ (hash, protocol, duplicate names, transfer tests)

5. Duplicate Filename Strategy

Never use the filename alone as the identity of a file. Generate a UUID for every transfer and save each received file in its own directory, for example received/<uuid-1>/report.pdf and received/<uuid-2>/report.pdf. Store the sender hash and other metadata against that UUID in SQLite. This prevents accidental overwrites and ensures each received file is compared with the correct expected hash.

6. Implementation Steps

Step 1 — Hashing: Implement chunked SHA-256 calculation and test it with empty, small, and large files.

Step 2 — Protocol: Implement length-prefixed metadata and a recv_exact helper because TCP may split data across reads.

Step 3 — Receiver: Build the TCP server; validate incoming metadata and file sizes; save to unique directories.

Step 4 — Sender: Select files, create UUIDs, calculate hashes, and send metadata followed by file bytes.

Step 5 — Verification: Calculate the receiver hash and compare it with the sender hash automatically; add manual verification.

Step 6 — Records and tests: Record transfer history in SQLite and test duplicate names, altered files, empty files, interruptions, and invalid metadata.

Step 7 — Security and documentation: Add authentication and TLS, document setup, and capture demonstration screenshots.

7. Testing Plan

Test	Expected result
Normal file transfer	Sender and receiver SHA-256 hashes match; status VERIFIED.
Two files with the same name	Both are stored separately and checked against their own transfer records.
Modify a received file	Manual hash check reports MISMATCH.
Interrupted or incomplete transfer	Status FAILED; incomplete file is not marked verified.
Empty and large files	Correct transfer and digest using chunked processing.
Unsafe path or malformed metadata	Request is rejected or handled safely.

8. Security Notes

SHA-256 comparison detects content differences only when the expected digest is trustworthy. A hash sent over an unauthenticated connection can be replaced by an attacker. For stronger protection, use TLS, authenticate senders, and consider signing transfer metadata with a private key. Validate file sizes and paths, restrict access to the receiver, and never mark partial files as verified.

9. Expected Outcome

A Python application that transfers files over a network, handles multiple files with identical filenames without overwriting them, automatically verifies received content, supports manual hash checking, and maintains a transfer history with clear verification results.